



UNIVERSIDAD NACIONAL
DE EDUCACIÓN A DISTANCIA

Escuela Técnica Superior de
Ingeniería Informática



UNIVERSIDAD
COMPLUTENSE DE MADRID

Facultad de
Informática

[TÍTULO DEL TRABAJO]

Martin Augusto Reigadas Teran

Director/a: [Director/a]

Co-director/a: [Co-director/a]

Trabajo de Fin de Máster

Máster Universitario
en Ingeniería de Control y Sistema

[Fecha de presentación/Convocatoria]

Agradecimientos

[Quisiera dedicar ...]

Resumen

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Palabras clave: palabra1, palabra2, palabra3

Abstract

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Keywords: keyword1, keyword2, keyword3

Índice general

1. Introducción	1
1.1. Motivación	1
1.2. Propuesta y objetivos	2
1.3. Estructura del documento	3
2. Estado del Arte	5
2.1. Algoritmos de Aprendizaje por Refuerzo para Control Reactivo	5
2.2. Percepción Sensorial Multimodal y Observabilidad Parcial	6
2.3. Aprendizaje por Currículo y Navegación Segura	6
3. Diseño y arquitectura	9
3.1. Formulación del Problema como POMDP	9
3.2. Arquitectura de control	10
3.3. Modelo Cinemático del AGV sobre Rejilla	11
3.3.1. Modelo con Giro y Avance (GridRobot)	11
3.3.2. Modelo sin Giro y Traslación Directa (GridRobotNotTurning)	11
3.4. Espacio de Observaciones Perceptuales Locales	12
3.5. Función de Recompensa Compuesta	13
3.6. Arquitecturas de Redes Neuronales DQN y PPO	13
4. Configuración Experimental y Análisis de Resultados	15
4.1. Configuración de los Escenarios de la Rejilla	15
4.2. Hiperparámetros de Entrenamiento de los Algoritmos	15
4.3. Espacio de Acciones Discreto	15
4.4. Función de Recompensa Compuesta	16
4.5. Mecanismo de Reinicio Suave (Soft Reset)	17
4.6. Arquitecturas de Redes Neuronales DQN y PPO	17
4.7. Arquitectura	17
5. Resultados	19
5.1. Configuración de los Escenarios de la Rejilla	19

5.2. Hiperparámetros de Entrenamiento de los Algoritmos	19
5.3. Métricas de Evaluación de Navegación	20
5.4. Estructura de las Figuras y Curvas de Aprendizaje	20
6. Conclusiones y trabajos futuros	23
Bibliografía y referencias	24
A. Ejemplos en $\text{\LaTeX}$	29
A.1. Referencias, citas y bibliografía	29
A.1.1. Referencias	29
A.1.2. Bibliografía	29
A.2. Figuras	30
A.3. Tablas	30
A.4. Código	31
A.5. Listas	32
A.6. Ecuaciones	32

Índice de figuras

3.1. Arquitectura de control DRL	10
3.2. Diagrama visual de conexiones de la arquitectura de la red SimpleQNetwork de control adaptada a los espacios de acción discretos del AGV ($ A = 3$ para <code>GridRobot</code> y $ A = 4$ para <code>GridRobotNotTurning</code>).	14
4.1. Diagrama visual de conexiones de la arquitectura de la red SimpleQNetwork de control de avance y giro del AGV.	17
5.1. Curvas de convergencia del entrenamiento de los agentes DRL en el entorno de rejilla simple estática.	21
5.2. Comparación visual en 2D de las trayectorias resultantes del AGV autónomo reactivo evadiendo obstáculos de rejilla.	21
A.1. Ejemplo de figura	30

Índice de tablas

3.1. Parámetros y dimensiones estructurales de las redes neuronales DQN y PPO implementadas.	14
4.1. Parámetros de entrenamiento del optimizador Adam y del buffer de experiencia.	16
4.2. Parámetros y dimensiones estructurales de las redes neuronales DQN y PPO implementadas.	18
5.1. Parámetros de entrenamiento del optimizador Adam y del buffer de experiencia.	19
5.2. Esqueleto comparativo de rendimiento de navegación y tasa de éxito (promedio sobre 200 episodios de evaluación).	20
5.3. Esqueleto comparativo de consumo energético del AGV y latencia del control reactivo en tiempo real.	20
A.1. Ejemplo de tabla	31

Capítulo 1

Introducción

1.1. Motivación

La automatización de los procesos intralogísticos es uno de los pilares de la transición hacia la manufactura flexible de la Industria 4.0 [Hu et al., 2020]. En este contexto, los Vehículos de Guiado Automático (AGVs, por sus siglas en inglés) desempeñan un papel fundamental en el transporte autónomo de componentes dentro de plantas industriales y almacenes [Xue et al., 2022a]. Tradicionalmente, la navegación de AGVs se ha fundamentado en planificadores de rutas clásicos que requieren información cartográfica exhaustiva previa (navegación basada en mapas, utilizando SLAM o guías físicas) [Sivashangaran and Eskandarian, 2023a]. Sin embargo, estas metodologías tradicionales conllevan elevados costes de instalación, mantenimiento y actualización del mapa cuando el entorno sufre alteraciones estructurales frecuentes, lo cual es habitual en entornos logísticos modernos de manufactura esbelta [Xue et al., 2022b].

Los métodos clásicos de planificación local de rutas en tiempo real, tales como los Campos Potenciales Artificiales (APF, por sus siglas en inglés), presentan la ventaja de poseer una estructura matemática simple y un bajo coste computacional, idóneos para controles reactivos rápidos [Pérez et al., 2022]. No obstante, sufren de limitaciones teóricas insalvables, destacando la alta propensión de quedar atrapados en situaciones de mínimos locales estables, donde la fuerza de atracción hacia el objetivo se cancela exactamente con la fuerza de repulsión de los obstáculos cercanos [Pérez et al., 2022]. Por otro lado, las heurísticas globales de búsqueda basadas en muestreo estocástico como RRT o en rejillas como A^* resultan computacionalmente costosas de recalcular en entornos dinámicos en tiempo real y son sensibles al ruido sensorial local [Oh et al., 2025].

Para solventar estas barreras tecnológicas, surge la navegación reactiva libre de mapas (*map-less navigation*) basada en el Aprendizaje por Refuerzo Profundo (DRL, por sus siglas en inglés) [Guo et al., 2025a]. Mediante DRL, es posible entrenar redes neuronales profundas que sirvan como políticas de control reactivo extremo a extremo (*end-to-end*), mapeando

directamente lecturas de sensores de rango locales (como nubes de puntos de LiDAR) y coordenadas relativas de la meta a comandos de actuación discretos o continuos [Sivashangaran et al., 2023]. Este paradigma elimina por completo la dependencia de mapas globales previos y posicionamiento SLAM robusto, lo cual dota al AGV de una robustez y flexibilidad excepcionales ante escenarios industriales impredecibles y estáticos con obstáculos no mapeados [Xue et al., 2022b].

1.2. Propuesta y objetivos

El presente trabajo de investigación propone el diseño, implementación y evaluación de un sistema de navegación reactiva map-less para un AGV operando en un entorno de rejilla discreta bidimensional con obstáculos estáticos desconocidos. La política de navegación reactiva del AGV procesa información perceptual multimodal ligera: un vector de sensado de LiDAR submuestreado en secciones regulares locales, el vector de posición relativa a la meta expresado en coordenadas polares (rumbo y distancia) y el estado de velocidad previa del robot. A partir de estas observaciones locales, la política entrenada con DRL genera decisiones sobre comandos discretos de dirección (avanzar, girar a la izquierda, girar a la derecha) con el fin de guiar al AGV de manera segura hacia la meta.

Para superar el severo problema de la dispersión de recompensa (*sparse reward*) inherente a las tareas de navegación de largo horizonte temporal en entornos con obstáculos, se formula una función de recompensa compuesta con penalizaciones por proximidad que moldea el comportamiento seguro del robot. Asimismo, se incorpora un Handler de Interrupciones en tiempo real (*is_stuck*) con un buffer de memoria temporal que activa un mecanismo de reinicio suave (*soft reset*) de la exploración estocástica cuando el AGV cae en una zona de bloqueo o mínimo local estático de la rejilla.

Objetivo General: Diseñar y evaluar una arquitectura de control de navegación reactiva libre de mapas (*map-less*) para un robot móvil tipo AGV en entornos discretos de rejilla con obstáculos estáticos utilizando algoritmos de Aprendizaje Profundo por Refuerzo (DRL).

Objetivos Específicos:

1. Formular matemáticamente el problema de navegación reactiva como un Proceso de Decisión de Markov Parcialmente Observable (POMDP) aplicable a entornos con sensores locales y sin mapa previo [Md et al., 2024].
2. Diseñar un vector de observaciones multimodal que combine lecturas discretizadas de LiDAR en secciones angulares y rumbo relativo a la meta para controlar la explosión dimensional del estado.
3. Desarrollar una función de recompensa compuesta que integre penalizaciones continuas de proximidad y un mecanismo heurístico de desatasco mediante reinicios suaves de exploración (*soft reset*).

4. Implementar y comparar las arquitecturas de red neuronal de aproximación de valor y política (como DQN y PPO) utilizando frameworks modernos basados en PyTorch.

Acrónimo	Definición
AGV	Vehículo de Guiado Automático (Automated Guided Vehicle)
DRL	Aprendizaje por Refuerzo Profundo (Deep Reinforcement Learning)
DQN	Red Q Profunda (Deep Q-Network)
PPO	Optimización de Política Próxima (Proximal Policy Optimization)
POMDP	Proceso de Decisión de Markov Parcialmente Observable

5. Validar el rendimiento de la política aprendida en diversos escenarios estáticos simulados de rejilla frente a los enfoques clásicos de Campos Potenciales Artificiales [Pérez et al., 2022].

1.3. Estructura del documento

La presente memoria de tesis de maestría se estructura de la siguiente manera:

- El **Capítulo 1 (Introducción)** contextualiza la motivación, los objetivos específicos y la propuesta del proyecto.
- El **Capítulo 2 (Estado del Arte)** revisa de manera exhaustiva la literatura científica existente sobre el uso de DRL en robótica móvil, comparando algoritmos de control, percepción multimodal y el aprendizaje curricular.
- El **Capítulo 3 (Diseño y arquitectura)** detalla formalmente el modelo matemático del POMDP, la arquitectura de control, la cinemática discreta del robot de rejilla, la estructura del LiDAR por sectores angulares, la función de recompensa y las arquitecturas de las redes DQN/PPO aplicadas.
- El **Capítulo 4 (Configuración Experimental y Análisis de Resultados)** introduce el esqueleto del escenario de evaluación, los hiperparámetros de entrenamiento de las redes y las métricas de rendimiento sugeridas para su futura cumplimentación de datos reales.
- El **Capítulo 5 (Conclusiones y Trabajo Futuro)** resume los aportes científicos fundamentales y propone líneas de investigación futuras.

Capítulo 2

Estado del Arte

2.1. Algoritmos de Aprendizaje por Refuerzo para Control Reactivo

El desarrollo del DRL aplicado al guiado de robots autónomos se divide primordialmente entre algoritmos de optimización de gradiente de política directos *on-policy* y algoritmos basados en valor de acción de tipo crítico-actor *off-policy* [Pérez et al., 2022]. Entre los enfoques *on-policy*, el algoritmo de Optimización de Política Próxima (PPO, por sus siglas en inglés) destaca por su extraordinaria estabilidad numérica y robustez ante el ajuste de hiperparámetros [Shakya et al., 2023]. PPO previene las actualizaciones de política excesivamente destructivas mediante una función de pérdida sustituta con restricciones por recorte del ratio de probabilidad de la acción, limitando la divergencia de Kullback-Leibler entre la política vieja y la nueva [Research, 2025]:

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t) \right] \quad (2.1)$$

No obstante, dado que PPO es *on-policy*, padece de una baja eficiencia en el uso de muestras, requiriendo millones de interacciones directas con el entorno de simulación para converger hacia una política robusta [Oh et al., 2025].

Para solventar la ineficiencia de muestreo, se han extendido aproximaciones *off-policy* de crítico-actor continuas como DDPG y su variante de críticos retrasados gemelos TD3 [Sivashangaran and Eskandarian, 2023a]. Adicionalmente, el algoritmo Soft Actor-Critic (SAC) destaca como un método altamente eficiente basado en la maximización de la entropía máxima, el cual optimiza de manera conjunta el retorno esperado y el grado de aleatoriedad de la política [Sivashangaran and Eskandarian, 2023b]. Esta adición fomenta una exploración muy agresiva y previene que el actor converja prematuramente a mínimos locales estáticos

o cuellos de botella subóptimos en la navegación [Xue et al., 2022b]:

$$J(\pi) = \sum_{t=0}^T \mathbb{E}_{(s_t, a_t) \sim \rho_\pi} [R(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot|s_t))] \quad (2.2)$$

En entornos logísticos de rejilla donde las acciones son intrínsecamente discretas (avanzar, girar), las redes Q profundas (DQN) de tipo *off-policy* y sus extensiones de doble estimación (Double DQN) resultan óptimas por su simplicidad de convergencia teórica y por permitir el reuso masivo de muestras previas del buffer de replay [Hu et al., 2020].

2.2. Percepción Sensorial Multimodal y Observabilidad Parcial

La navegación reactiva sin mapas se enmarca típicamente bajo la formulación matemática de un Proceso de Decisión de Markov Parcialmente Observable (POMDP), dado que el AGV solo dispone de lecturas de sensores de rango locales y carece de la posición global de los obstáculos en un plano cartesiano unificado [Md et al., 2024]. El sensado local con LiDARs discretos o continuos presenta problemas críticos de oclusión geométrica (puntos ciegos detrás de paredes) y ruidos sensoriales de rebote estocástico [Sivashangaran et al., 2023]. Para atenuar la dimensionalidad de las observaciones crudas de LiDAR (que pueden llegar a cientos de rayos densos), la literatura propone técnicas de discretización modular o compresión mediante autoencoders geométricos robustos (GDAE) [Guo et al., 2025b].

Por otra parte, la fusión de información sensorial visual profunda con datos de rango LiDAR se consolida mediante el uso de redes convolucionales preentrenadas (como MobileNetV3) en esquemas autoasociativos estáticos para codificar el entorno visual en un vector de baja dimensionalidad espacial [Research, 2025, Xue et al., 2022b]. No obstante, la pérdida de observabilidad temporal en el POMDP requiere el uso de memorias recurrentes profundas, tales como celdas LSTM o arquitecturas autorregresivas basadas en Transformers de decisión (Decision Transformers) [Ge et al., 2024a,b]. Estos componentes de memoria temporal permiten procesar una ventana histórica de observaciones y acciones previas $h_t = (o_1, a_1, \dots, o_t)$, extrayendo el “estado de creencia” estocástico implícito (*belief state*) indispensable para desambiguar estados geoméricamente idénticos y evadir de forma activa obstáculos dinámicos u oclusiones severas [Md et al., 2024, Oh et al., 2025].

2.3. Aprendizaje por Currículo y Navegación Segura

El entrenamiento de un AGV map-less desde cero bajo un esquema tradicional de DRL se enfrenta al cuello de botella de la “recompensa dispersa” (*sparse reward*) [Ye et al., 2023]. Si el espacio de estados es grande y la meta es de dimensiones reducidas en comparación con

la arena de pruebas, la probabilidad estocástica de que el agente alcance la meta mediante exploración aleatoria pura al inicio del entrenamiento es prácticamente nula, lo que provoca estancamientos en el gradiente de optimización [Oh et al., 2025]. Para mitigar esto, se ha introducido el Aprendizaje por Currículo Automático (ACL, por sus siglas en inglés) [Xue et al., 2022b].

La metodología propuesta por el algoritmo *NavACL* y su versión extendida con retroalimentación del valor de acción de los críticos, *NavACL-Q*, proponen dinámicamente escenarios de entrenamiento de dificultad adaptativa progresiva [Xue et al., 2022b]. *NavACL-Q* evalúa métricas geométricas de los mapas logísticos (distancia geodésica, holgura del agente y del objetivo ante obstáculos, y el ángulo de rumbo inicial) combinándolas con la estimación de valor Q predicha por la red crítica del agente, para proponer poses iniciales que se sitúen en la frontera del conocimiento actual de la política [Xue et al., 2022a]. A medida que el AGV incrementa su tasa de éxito local, el currículo de forma automática expande la distancia inicial de navegación y la complejidad de los laberintos. Adicionalmente, el Aprendizaje por Refuerzo Seguro (Safe RL) incorpora la teoría de Optimización de Política con Restricciones (CPO, por sus siglas en inglés), donde la función de costo modela explícitamente el riesgo de colisión catastrófica, asegurando que la política explore el espacio con garantías de seguridad física bajo restricciones de barreras de control [Gao et al., 2025a].

Capítulo 3

Diseño y arquitectura

En este capítulo se describe de manera detallada la metodología propuesta para resolver el problema de navegación autónoma reactiva de un Vehículo de Guiado Automático (AGV) en entornos de rejilla con obstáculos estáticos. En primer lugar, se formaliza el problema de navegación map-less bajo el marco teórico de los Procesos de Decisión de Markov Parcialmente Observables (POMDP). Posteriormente, se exponen los modelos cinemáticos discretos implementados (para los robots GridRobot y GridRobotNotTurning), describiendo las transiciones paso a paso por iteración, así como el procesamiento del sensor GridLidar por secciones para controlar la dimensionalidad del estado. Finalmente, se detalla la formulación matemática de la función de recompensa compuesta asimétrica y la estructura de las redes neuronales profundas (DQN y PPO) utilizadas por los agentes.

3.1. Formulación del Problema como POMDP

Dado que el AGV no tiene acceso global al plano de los obstáculos estáticos de la rejilla, el proceso se modela rigurosamente como un Proceso de Decisión de Markov Parcialmente Observable (POMDP), definido por la tupla matemática $\langle S, A, T, R, \Omega, O, \gamma \rangle$ [Md et al., 2024], donde:

- S es el espacio de estados del sistema que incluye la pose real absoluta del AGV y la configuración de los obstáculos de la rejilla.
- A es el espacio discreto de acciones ejecutables por el robot de guiado automático, el cual varía de acuerdo con el modelo cinemático implementado.
- $T(s_{t+1}|s_t, a_t)$ es la función estocástica de transición de estados de la rejilla.
- $R(s_t, a_t)$ es la recompensa asimétrica compuesta que guía y penaliza al agente.
- Ω representa el espacio de observaciones locales perceptuales del robot.

- $O(o_t|s_t, a_{t-1})$ denota la función de distribución probabilística de la observación sensorial local.
- $\gamma \in [0, 1)$ es el factor de descuento temporal que pondera la ganancia futura.

Ante la falta de observabilidad completa del estado geométrico s_t , el agente estima inductivamente un estado de creencia o vector de confianza $b_t(s) \in \mathcal{B}$ sobre la distribución del estado real mediante actualización recursiva bayesiana [Md et al., 2024]:

$$b_{t+1}(s') = \eta O(o_{t+1}|s', a_t) \sum_{s \in S} T(s'|s, a_t) b_t(s) \quad (3.1)$$

donde η es una constante normalizadora de probabilidad [Md et al., 2024].

3.2. Arquitectura de control

La arquitectura de control se basa en un entorno matricial en el que el robot se desplaza o rota secuencialmente por iteración. Este entorno envía el vector de observación (o_t), que describe el estado del robot y su entorno inmediato, al agente DRL para la selección de la acción discreta (a_t).

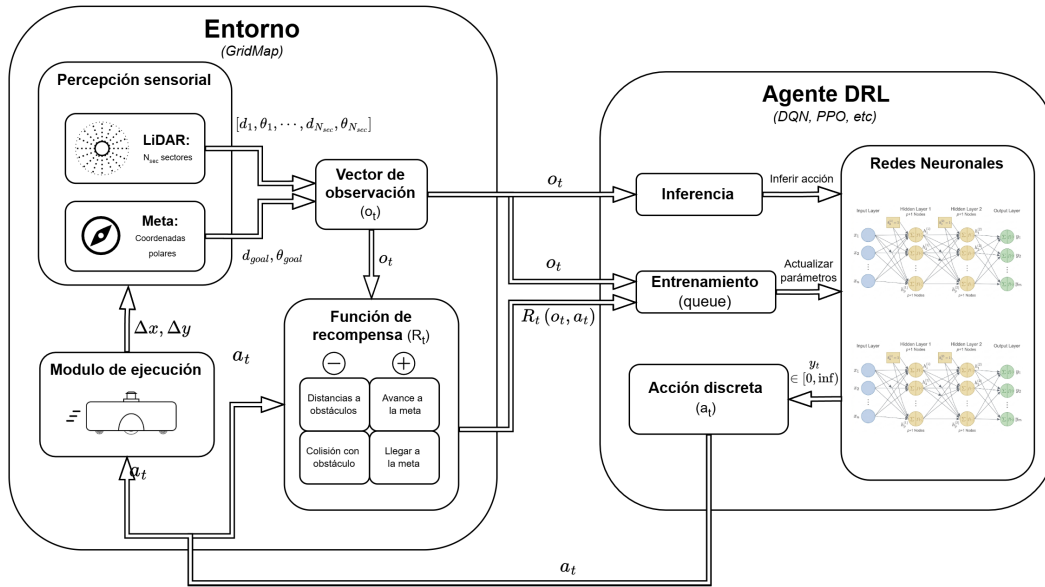


Figura 3.1: Diagrama de la arquitectura de control DRL y flujo de información entre el entorno y el agente.

Como se observa en la figura 3.1, las observaciones (o_t) y la señal de recompensa ($R_t(o_t, a_t)$) se almacenan en una cola de experiencia para el posterior entrenamiento de las redes neuronales del agente.

3.3. Modelo Cinemático del AGV sobre Rejilla

El vehículo de guiado automático (AGV) opera sobre una cuadrícula discreta de celda fija $\Delta_{\text{cell}} = 10$ unidades de simulación. Y se formalizan dos modelos cinemáticos alternativos en espacio discreto.

3.3.1. Modelo con Giro y Avance (GridRobot)

El estado del AGV en el instante t se define por el vector de pose $\mathbf{x}_t = [x_t, y_t, \theta_t]^T \in \mathcal{X}$, con $\mathcal{X} = \mathbb{Z}^2 \times \Theta$ y $\Theta = \{0^\circ, 90^\circ, 180^\circ, 270^\circ\}$. La transición de estado dada la acción $a_t \in \{0, 1, 2\}$ se rige por la ecuación diferencial discreta:

$$\mathbf{x}_{t+1} = \mathbf{x}_t + \mathbf{f}(\mathbf{x}_t, a_t) \quad (3.2)$$

donde el operador de transición $\mathbf{f}(\mathbf{x}_t, a_t)$ se define vectorialmente como:

$$\mathbf{f}(\mathbf{x}_t, a_t) = \begin{bmatrix} \delta(a_t) \mathbf{d}(\theta_t) \\ \Delta\theta(a_t) \end{bmatrix} \quad (3.3)$$

Siendo $\mathbf{d}(\theta_t) = [\cos \theta_t, \sin \theta_t]^T \in \{-1, 0, 1\}^2$ el vector unitario de dirección cardinal, $\delta(a_t) = \mathbb{I}(a_t = 0)$ la función indicadora de traslación, y $\Delta\theta(a_t)$ el incremento angular modelado como:

$$\Delta\theta(a_t) = \begin{cases} 0^\circ & \text{si } a_t = 0 \text{ (Avanzar)} \\ +90^\circ \text{ (mód } 360^\circ) & \text{si } a_t = 1 \text{ (Giro Izquierda)} \\ -90^\circ \text{ (mód } 360^\circ) & \text{si } a_t = 2 \text{ (Giro Derecha)} \end{cases} \quad (3.4)$$

3.3.2. Modelo sin Giro y Traslación Directa (GridRobotNotTurning)

Para este modelo puramente holonómico discreto, la orientación se asume constante, reduciendo el espacio de estados a la posición cartesiana $\mathbf{p}_t = [x_t, y_t]^T \in \mathbb{Z}^2$. La ecuación de movimiento adopta la forma matricial compacta:

$$\mathbf{p}_{t+1} = \mathbf{p}_t + \mathbf{M} \mathbf{e}_{a_t} \quad (3.5)$$

donde $\mathbf{e}_{a_t} \in \mathbb{R}^4$ es el vector canónico unitario de la acción $a_t \in \{0, 1, 2, 3\}$, y $\mathbf{M} \in \mathbb{Z}^{2 \times 4}$ es la matriz de desplazamiento cardinal:

$$\mathbf{M} = \begin{bmatrix} 0 & 0 & 1 & -1 \\ -1 & 1 & 0 & 0 \end{bmatrix} \quad \text{correspondiente a } \{\text{Norte, Sur, Este, Oeste}\} \quad (3.6)$$

3.4. Espacio de Observaciones Perceptuales Locales

El estado observable por el agente en cada instante t , denotado como $\mathbf{o}_t \in \mathbb{R}^{D_{\text{obs}}}$, se estructura como un vector denso formado por la concatenación de la información exteroceptiva del sensor LiDAR y la odometría relativa hacia la meta. Inicialmente, este vector se define como:

$$\mathbf{o}_t = \begin{bmatrix} \mathbf{r}_{\text{LiDAR}} \\ \boldsymbol{\theta}_{\text{LiDAR}} \\ d_{\text{goal}} \\ \theta_{\text{goal}} \end{bmatrix} \in \mathbb{R}^{2N_{\text{sec}}+2} \quad (3.7)$$

donde $\mathbf{r}_{\text{LiDAR}}, \boldsymbol{\theta}_{\text{LiDAR}} \in \mathbb{R}^{N_{\text{sec}}}$ representan los vectores de distancias mínimas por sector y sus ángulos de incidencia asociados, respectivamente. Por su parte, d_{goal} y θ_{goal} indican la distancia cartesiana euclídea y el ángulo relativo desde el AGV hasta la meta. La lectura de distancia no normalizada para cada sector k se obtiene mediante:

$$r_{\text{LiDAR}}^{(k)} = \min_{\phi \in \text{Sector}_k} \{d_{\text{ray}}(\phi)\}, \quad k \in \{1, \dots, N_{\text{sec}}\} \quad (3.8)$$

Para abordar la pérdida de la propiedad de Markov inherente a la observabilidad parcial del entorno (POMDP), el espacio de observaciones se amplía incluyendo la memoria inercial de la última acción ejecutada, a_{t-1} . Adicionalmente, con el objetivo de estabilizar el cálculo de gradientes y acelerar la convergencia de la red neuronal, las magnitudes espaciales se normalizan al rango $[0, 1]$. La formulación final de la ecuación (3.7) resulta en:

$$\mathbf{o}_t = \begin{bmatrix} \bar{\mathbf{r}}_{\text{LiDAR}} \\ \boldsymbol{\theta}_{\text{LiDAR}} \\ \bar{d}_{\text{goal}} \\ \theta_{\text{goal}} \\ a_{t-1} \end{bmatrix} \in \mathbb{R}^{2N_{\text{sec}}+3} \quad (3.9)$$

donde $\bar{\mathbf{r}}_{\text{LiDAR}}$ contiene las distancias sectoriales normalizadas respecto al rango máximo del sensor d_{max} , calculadas como:

$$\bar{r}_{\text{LiDAR}}^{(k)} = \min_{\phi \in \text{Sector}_k} \left\{ \frac{d_{\text{ray}}(\phi)}{d_{\text{max}}} \right\}, \quad k \in \{1, \dots, N_{\text{sec}}\} \quad (3.10)$$

De manera análoga, $\bar{d}_{\text{goal}}$ representa la distancia cartesiana a la meta normalizada mediante d_{diag} , correspondiente a la diagonal del mapa cartográfico (distancia máxima franqueable en la rejilla):

$$\bar{d}_{\text{goal}} = \frac{d_{\text{goal}}}{d_{\text{diag}}} \quad (3.11)$$

3.5. Función de Recompensa Compuesta

La recompensa asimétrica compuesta $R(s_t, a_t)$ asocia incentivos dispersos de finalización y penalizaciones estocásticas continuas de seguridad para evitar colisiones [?]:

$$R(s_t, a_t) = R_{terminal} + \mathcal{R}_{progreso} - R_{time} - \mathcal{P}_{proximidad} \quad (3.12)$$

donde:

- $R_{terminal}$ es la recompensa estocástica de estados de parada terminales:

$$R_{terminal} = \begin{cases} +100,0, & \text{si AGV alcanza meta (éxito)} \\ -100,0, & \text{si colisiona con obstáculo} \\ -100,0, & \text{si excede límite temporal de pasos } T_{max} \end{cases} \quad (3.13)$$

- $\mathcal{R}_{progreso} = w_d \cdot (d_{t-1} - d_t)$ premia la aproximación radial real hacia la meta de destino.
- $R_{time} = 0,1$ es una penalización constante por paso de tiempo para forzar al AGV a tomar rutas directas y cortas [Guo et al., 2025b].
- $\mathcal{P}_{proximidad}$ es la penalización de seguridad activa si el AGV entra dentro de la zona crítica definida por el radio de seguridad de la rejilla ($\mathcal{D}_{safety} = d_{max} \cdot \frac{2}{3}$) [?]:

$$\mathcal{P}_{proximidad} = \begin{cases} \mathcal{P}_{max} \cdot \left(1,0 - \frac{d_{min}}{\mathcal{D}_{safety}}\right), & \text{si } d_{min} < \mathcal{D}_{safety} \\ 0,0, & \text{en otro caso} \end{cases} \quad (3.14)$$

fijando la penalización máxima en $\mathcal{P}_{max} = 10^{-2}$ (0,01).

3.6. Arquitecturas de Redes Neuronales DQN y PPO

Las aproximaciones de valor y políticas del actor-crítico se estiman mediante perceptrones multicapa totalmente conectados (MLP) cuyas capas y dimensiones se detallan en el Cuadro 4.2 y se visualizan estructuralmente en la Figura 4.1 [?]. Estas estructuras se adaptan al modelo cinemático activo:

Tabla 3.1: Parámetros y dimensiones estructurales de las redes neuronales DQN y PPO implementadas.

Red Neuronal	Capa / Tipo	Dimensión (Nodos)	Activación	Parámetro del Código
SimpleQNetwork citepnetworks.txt (Agente DQN)	Entrada	$D_{obs} = 12$	Ninguna	Observaciones del sensor
	Oculto 1 (Linear)	253	ReLU	Extracción de características
	Oculto 2 (Linear)	56	ReLU	Homogeneización dimensional
	Salida (Linear)	3/4	Ninguna	Valores Q de acciones discretas
ValueNetwork citepnetworks.txt (Crítico PPO)	Entrada	$D_{obs} = 12$	Ninguna	Vector de estado percibido
	Oculto 1 (Linear)	$H = 64$	ReLU	Dimensión oculta del crítico
	Oculto 2 (Linear)	64	ReLU	Representación de valores
	Salida (Linear)	1	Ninguna	Estimación de valor $V(s)$

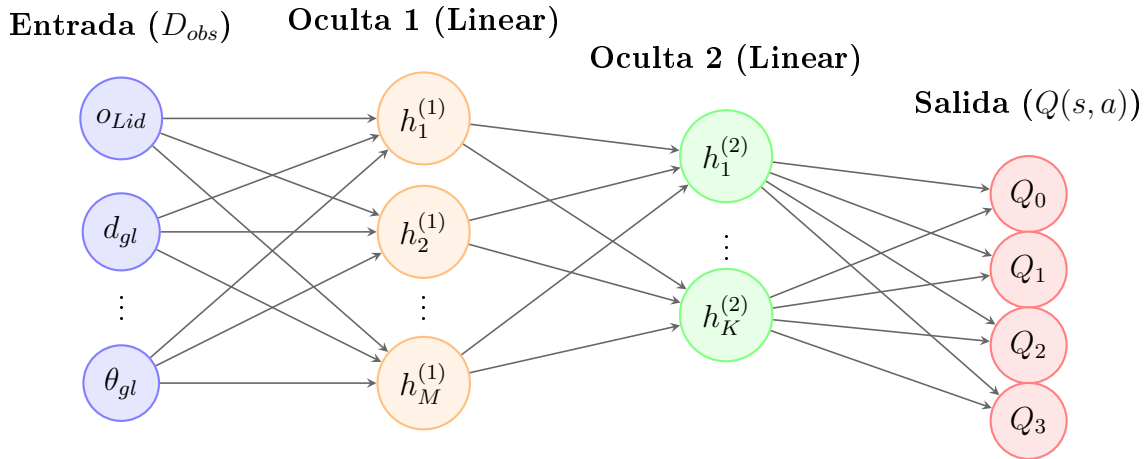


Figura 3.2: Diagrama visual de conexiones de la arquitectura de la red SimpleQNetwork de control adaptada a los espacios de acción discretos del AGV ($|A| = 3$ para **GridRobot** y $|A| = 4$ para **GridRobotNotTurning**).

Capítulo 4

Configuración Experimental y Análisis de Resultados

4.1. Configuración de los Escenarios de la Rejilla

Los experimentos numéricos y la evaluación de robustez se llevan a cabo sobre tres configuraciones de mapas estáticos cargados dinámicamente desde archivos de configuración serializados JSON [??]:

- `map_obstacle_free.json`: Escenario libre de obstáculos estáticos para validar la velocidad de convergencia teórica del AGV hacia la meta de navegación [?].
- `map_simple.json`: Escenario con un obstáculo geométrico rectangular central de rejilla fija para validar la maniobrabilidad básica de evasión de esquinas [?].
- `map_1_obstacle.json`: Escenario con un obstáculo asimétrico para validar el escape reactivo estocástico implementado [?].

4.2. Hiperparámetros de Entrenamiento de los Algoritmos

El entrenamiento de los agentes DQN y PPO se implementa utilizando los hiperparámetros estandarizados recopilados de tus scripts en el Cuadro 5.1 [??].

4.3. Espacio de Acciones Discreto

De acuerdo con el desarrollo lógico del software en el script `agent.txt`, el espacio de acciones se discretiza en tres decisiones de control de baja latencia para la navegación sobre la cuadrícula [?]:

Tabla 4.1: Parámetros de entrenamiento del optimizador Adam y del buffer de experiencia.

Hiperparámetro	Valor DQN / PPO	Descripción Técnica en el Código
Tasa de Aprendizaje (α_{lr})	10^{-3}	Tasa del optimizador Adam de la red de política
Factor de Descuento (γ)	0,99	Tasa de descuento de ganancias futuras [?]
Tamaño de Lote (<i>Batch size</i>)	64	Tamaño de muestra del optimizador estocástico
Pasos de Decaimiento (ϵ_{decay})	1,000,000	Pasos del decaimiento extendido de exploración
Capacidad del Buffer	100,000	Tamaño límite del buffer de replay de transición
Norma de Gradiente Máxima	5,0	Límite del recorte del gradiente (<i>gradient clipping</i>)

- **Acción 0 (Avanzar):** Comando para avanzar en línea recta a velocidad constante ($v = 1,0$ celda/paso, $\omega = 0,0$ rad/s).
- **Acción 1 (Girar Izquierda):** Comando de rotación pura a la izquierda ($v = 0,0$, $\omega = +\pi/2$ rad/s).
- **Acción 2 (Girar Derecha):** Comando de rotación pura a la derecha ($v = 0,0$, $\omega = -\pi/2$ rad/s).

4.4. Función de Recompensa Compuesta

La recompensa asimétrica compuesta $R(s_t, a_t)$ asocia incentivos dispersos de finalización y penalizaciones estocásticas continuas de seguridad para evitar colisiones [?]:

$$R(s_t, a_t) = R_{terminal} + \mathcal{R}_{progreso} - R_{time} - \mathcal{P}_{proximidad} \quad (4.1)$$

donde:

- $R_{terminal}$ es la recompensa estocástica de estados de parada terminales:

$$R_{terminal} = \begin{cases} +100,0, & \text{si AGV alcanza meta (éxito)} \\ -100,0, & \text{si colisiona con obstáculo} \\ -100,0, & \text{si excede límite temporal de pasos } T_{max} \end{cases} \quad (4.2)$$

- $\mathcal{R}_{progreso} = w_d \cdot (d_{t-1} - d_t)$ premia la aproximación radial real hacia la meta de destino.
- $R_{time} = 0,1$ es una penalización constante por paso de tiempo para forzar al AGV a tomar rutas directas y cortas [Guo et al., 2025b].
- $\mathcal{P}_{proximidad}$ es la penalización de seguridad activa si el AGV entra dentro de la zona

crítica definida por el radio de seguridad de la rejilla ($\mathcal{D}_{safety} = d_{max} \cdot \frac{2}{3}$) [?]:

$$\mathcal{P}_{proximidad} = \begin{cases} \mathcal{P}_{max} \cdot \left(1, 0 - \frac{d_{min}}{\mathcal{D}_{safety}}\right), & \text{si } d_{min} < \mathcal{D}_{safety} \\ 0, 0, & \text{en otro caso} \end{cases} \quad (4.3)$$

siguiendo los hiperparámetros del código real de entrenamiento, fijando la penalización máxima en $\mathcal{P}_{max} = 10^{-2}$ (0,01) [?].

4.5. Mecanismo de Reinicio Suave (Soft Reset)

Para prevenir los frecuentes atrapamientos y estancamientos de la política en zonas de pasillos estrechos o esquinas estáticas (mínimos locales de la rejilla), se define el manejador de interrupciones `InterruptHandler` [?]. El sistema monitorea continuamente una cola de doble extremo (`deque`) de tamaño fijo de las últimas acciones y recompensas acumuladas [?]. La condición de atasco se determina heurísticamente mediante la función `is_stuck`:

$$\text{is_stuck} = \text{std}(\mathbf{x}_{last_30}) < \epsilon_{threshold} \quad \wedge \quad a_t \in \{1, 2\} \quad (4.4)$$

Si el robot se encuentra oscilando o atascado, el manejador ejecuta de manera reactiva un reinicio suave del escenario de exploración (`do_soft_reset`) [?]. Este algoritmo purga el buffer de memoria temporal del replay e incrementa temporalmente el factor de exploración estocástica ϵ del agente a un valor alto ($\epsilon \leftarrow 0,8$) para forzar al AGV a escapar del obstáculo de la rejilla de forma reactiva sin interrumpir drásticamente la continuidad del proceso de aprendizaje [??].

4.6. Arquitecturas de Redes Neuronales DQN y PPO

Las aproximaciones de valor y políticas del actor-crítico se estiman mediante perceptrones multicapa totalmente conectados (MLP) cuyas capas y dimensiones se detallan en el Cuadro 4.2 y se visualizan estructuralmente en la Figura 4.1 [?].

Figura 4.1: Diagrama visual de conexiones de la arquitectura de la red SimpleQNetwork de control de avance y giro del AGV.

4.7. Arquitectura

Antes de empezar de ver los modelos,

Tabla 4.2: Parámetros y dimensiones estructurales de las redes neuronales DQN y PPO implementadas.

Red Neuronal	Capa / Tipo	Dimensión (Nodos)	Activación	Parámetro del Código
SimpleQNetwork (Agente DQN)	Entrada	$D_{obs} = 12$	Ninguna	Observaciones del sensor
	Oculto 1 (Linear)	253	ReLU	Extracción de características
	Oculto 2 (Linear)	56	ReLU	Homogeneización dimensional
	Salida (Linear)	3	Ninguna	Valores Q de acciones candidatas
ValueNetwork (Crítico PPO)	Entrada	$D_{obs} = 12$	Ninguna	Vector de estado percibido
	Oculto 1 (Linear)	$H = 64$	ReLU	Dimensión oculta del crítico
	Oculto 2 (Linear)	64	ReLU	Representación de valores
	Salida (Linear)	1	Ninguna	Estimación de valor $V(s)$

Capítulo 5

Resultados

5.1. Configuración de los Escenarios de la Rejilla

Los experimentos numéricos y la evaluación de robustez se llevan a cabo sobre tres configuraciones de mapas estáticos cargados dinámicamente desde archivos de configuración serializados JSON [??]:

- `map_obstacle_free.json`: Escenario libre de obstáculos estáticos para validar la velocidad de convergencia teórica del AGV hacia la meta de navegación [?].
- `map_simple.json`: Escenario con un obstáculo geométrico rectangular central de rejilla fija para validar la maniobrabilidad básica de evasión de esquinas [?].
- `map_1_obstacle.json`: Escenario con un obstáculo asimétrico para validar el escape reactivo estocástico implementado [?].

5.2. Hiperparámetros de Entrenamiento de los Algoritmos

El entrenamiento de los agentes DQN y PPO se implementa utilizando los hiperparámetros estandarizados recopilados de tus scripts en el Cuadro 5.1 [??].

Tabla 5.1: Parámetros de entrenamiento del optimizador Adam y del buffer de experiencia.

Hiperparámetro	Valor DQN / PPO	Descripción Técnica en el Código
Tasa de Aprendizaje (α_{lr})	10^{-3}	Tasa del optimizador Adam de la red de políticas
Factor de Descuento (γ)	0,99	Tasa de descuento de ganancias futuras [?]
Tamaño de Lote (<i>Batch size</i>)	64	Tamaño de muestra del optimizador estocástico
Pasos de Decaimiento (ϵ_{decay})	1,000,000	Pasos del decaimiento extendido de exploración
Capacidad del Buffer	100,000	Tamaño límite del buffer de replay de transición
Norma de Gradiente Máxima	5,0	Límite del recorte del gradiente (<i>gradient clip</i>)

5.3. Métricas de Evaluación de Navegación

Para la validación objetiva del rendimiento del AGV libre de mapas frente a baselines clásicos, se estructuran las siguientes métricas de evaluación en el Cuadro 5.2 y Cuadro 5.3 [Gao et al., 2025b].

Tabla 5.2: Esqueleto comparativo de rendimiento de navegación y tasa de éxito (promedio sobre 200 episodios de evaluación).

Algoritmo / Política	SR (%)	SRN (%)	CT (prom.)	SPL
Campos Potenciales Clásicos (APF) [P'erez et al., 2022]	0.00	0.00	0.00	0.00
Algoritmos Genéticos (AG-MLP) [P'erez et al., 2022]	0.00	0.00	0.00	0.00
DQN (GridAgentNet Básica)	0.00	0.00	0.00	0.00
DQN con Soft Reset (GridAgent_2_FIXED)	0.00	0.00	0.00	0.00
PPO Reactivo (Map-less)	0.00	0.00	0.00	0.00

Tabla 5.3: Esqueleto comparativo de consumo energético del AGV y latencia del control reactivo en tiempo real.

Algoritmo / Política	Pasos por Episodio	Consumo de Energía (E_{total})
Campos Potenciales Clásicos (APF)	—	—
Algoritmos Genéticos (AG-MLP)	0.00	—
DQN (GridAgentNet Básica)	0.00	0.00
DQN con Soft Reset (GridAgent_2_FIXED)	0.00	0.00
PPO Reactivo (Map-less)	0.00	0.00

5.4. Estructura de las Figuras y Curvas de Aprendizaje

Para el futuro vaciado de las curvas e imágenes del simulador de rejilla, se preparan las siguientes maquetaciones de figuras de LaTeX:

[Espacio reservado para las curvas de convergencia de recompensa acumulada vs pasos de entrenamiento de DQN, DQN con Soft Reset y PPO]

Figura 5.1: Curvas de convergencia del entrenamiento de los agentes DRL en el entorno de rejilla simple estática.

[Espacio reservado para las trayectorias espaciales 2D sobre la rejilla GridMap mostrando los puntos de colisión del APF y las trayectorias seguras logradas por DRL]

Figura 5.2: Comparación visual en 2D de las trayectorias resultantes del AGV autónomo reactivo evadiendo obstáculos de rejilla.

Capítulo 6

Conclusiones y trabajos futuros

Bibliografía

- Hao Hu, Xiaoliang Jia, Qixuan He, Shifeng Fu, and Kuo Liu. Deep reinforcement learning based agvs real-time scheduling with mixed rule for flexible shop floor in industry 4.0. *Computers & Industrial Engineering*, 149:106749, 2020. doi: 10.1016/j.cie.2020.106749.
- Honghu Xue, Benedikt Hein, Mohamed Bakr, Georg Schildbach, Bengt Abel, and Elmar Rueckert. Using deep reinforcement learning with automatic curriculum learning for mapless navigation in intralogistics. *Applied Sciences*, 12(6):3153, 2022a. doi: 10.3390/app12063153.
- Shathushan Sivashangaran and Azim Eskandarian. Deep reinforcement learning for autonomous ground vehicle exploration without a-priori maps. *Advances in Artificial Intelligence and Machine Learning*, 3(2):1198–1219, 2023a. arXiv preprint arXiv:2301.04036.
- Honghu Xue, Benedikt Hein, Mohamed Bakr, Georg Schildbach, Bengt Abel, and Elmar Rueckert. Using deep reinforcement learning with automatic curriculum learning for mapless navigation in intralogistics. MDPI Applied Sciences, 2022b.
- David Abad P'erez, Basil Mohammed Al-Hadithi, and Victor Cadix Mart'in. Navigation of mobile robots using neural networks and genetic algorithms. *IEEE Latin American Transactions*, 2022.
- David Abad P'erez, Basil Mohammed Al-Hadithi, and V'ictor Cadix Mart'in. Navigation of mobile robots using neural networks and genetic algorithms. *IEEE Latin American Transactions*, 2022. URL <https://latamt.ieeeer9.org/index.php/transactions/article/view/8506>.
- Junhyuk Oh, David Silver, et al. Discovering deep reinforcement learning rules. *Nature Research article_1743243135*, 2025.
- Dayu Guo, Yuan Zhu, and Ke Lu. Map-less navigation algorithm for autonomous vehicles based on deep reinforcement learning. *Advances in Computer, Signals and Systems*, 9(1): 123–130, 2025a. doi: 10.23977/acss.2025.090117.

- Shathushan Sivashangaran, Apoorva Khairnar, and Azim Eskandarian. Autovrl: A high fidelity autonomous ground vehicle simulator for sim-to-real deep reinforcement learning. *IFAC-PapersOnLine*, 56(3):475–480, 2023. doi: 10.1016/j.ifacol.2023.12.069.
- Abdul Quadir Md, Dibyanshu Jaiswal, Senthilkumar Mohan, Nisreen Innab, Riza Sulaiman, Mohammed Kbiri Alaoui, and Ali Ahmadian. A novel approach for self-driving car in partially observable environment using life long reinforcement learning. *Sustainable Energy, Grids and Networks*, 38:101356, 2024. doi: 10.1016/j.segan.2024.101356.
- Ashish Kumar Shakya, Gopinatha Pillai, and Sohom Chakrabarty. Reinforcement learning algorithms: A brief survey. *Expert Systems with Applications*, 2023.
- Action Space Reduction Research. Action space reduction strategies for reinforcement learning in autonomous driving. Preprint arXiv:2507.05251, 2025.
- Shathushan Sivashangaran and Azim Eskandarian. Deep reinforcement learning for autonomous ground vehicle exploration without a-priori maps. *Advances in Artificial Intelligence*, 2023b.
- Y. Guo, X. Zhu, and Y. Lu. Map-less navigation algorithm for autonomous vehicles based on deep reinforcement learning. *Advances in Computer, Signals and Systems*, 2025b.
- Lun Ge, Xiaoguang Zhou, Yongqiang Li, and Yongcong Wang. Deep reinforcement learning navigation via decision transformer in autonomous driving. *Frontiers in Neurorobotics*, 18:1338189, 2024a. doi: 10.3389/fnbot.2024.1338189.
- Lun Ge, Xiaoguang Zhou, Yongqiang Li, and Yongcong Wang. Deep reinforcement learning navigation via decision transformer in autonomous driving. *Frontiers in Neurorobotics*, 2024b.
- Xianfeng Ye, Zhiyun Deng, Yanjun Shi, and Weiming Shen. Toward energy-efficient routing of multiple agvs with multi-agent reinforcement learning. *Sensors*, 23(12):5615, 2023. doi: 10.3390/s23125615.
- J. Gao, X. Pang, Q. Liu, and Y. Li. Hierarchical reinforcement learning for safe mapless navigation with congestion estimation. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025a.
- Jianqi Gao, Xizheng Pang, Qi Liu, and Yanjie Li. Hierarchical reinforcement learning for safe mapless navigation with congestion estimation. *ICRA 2025 Accepted Paper*, 2025b.
- Danilo Dessi, Francesco Osborne, Diego Reforgiato Recupero, Davide Buscaldi, Enrico Motta, and Harld Sack. Ai-kg: an automatically generated knowledge graph of artificial intelligence, 2020.

- P. Chen, Y. Lu, V. W. Zheng, X. Chen, and B. Yang. Knowedu: A system to construct knowledge graph for education. *IEEE Access*, 6:31553–31563, 2018. doi: 10.1109/ACCESS.2018.2839607.
- Yuehua Qin, Han Cao, and Leyi Xue. Research and application of knowledge graph in teaching: Take the database course as an example. *Journal of Physics: Conference Series*, 1607:012127, aug 2020. doi: 10.1088/1742-6596/1607/1/012127. URL <https://doi.org/10.1088/1742-6596/1607/1/012127>.
- Shaoxiong Ji, Shirui Pan, Erik Cambria, Pekka Marttinen, and Philip S. Yu. A survey on knowledge graphs: Representation, acquisition and applications, 2020.

Apéndice A

Ejemplos en L^AT_EX

Este primer apéndice presenta ejemplos en L^AT_EX de cómo incluir referencias, citas bibliográficas, figuras, tablas o código. Este apéndice se deberá eliminar de la memoria antes de entregar el trabajo.

A.1. Referencias, citas y bibliografía

A.1.1. Referencias

Para poder referenciar un elemento dentro de la memoria hay que marcarlo con una etiqueta (`\label{[id]}`) que lo identifique inequívocamente. Es habitual utilizar identificadores representativos, por ejemplo, para marcar la introducción podemos utilizar una etiqueta como `\label{chap:introduccion}`

Una vez marcado el elemento (capítulo, sección, figura, tabla, ...) en L^AT_EX, utilizaremos el comando `ref` indicando a qué etiqueta queremos referenciar (`\ref{[id]}`). Por ejemplo, de esta manera podemos referenciar al capítulo de la introducción con 1. Podemos acompañarlo de un texto como “...como se vió en el capítulo 1...” o bien utilizar el comando `\autoref{[i]}`, que incluiría el tipo del elemento y aparecería en el texto como Capítulo 1, o incluso referenciarlo por el nombre del elemento con `\nameref{[id]}`, lo que se mostraría como Introducción.

A.1.2. Bibliografía

Para añadir una cita bibliográfica al documento tendremos que asegurarnos que en el fichero de bibliografía (bibliografía.bib) se encuentre la entrada bibliográfica correspondiente. Una vez que tengamos nuestra entrada bibliográfica utilizaremos el comando de cita de L^AT_EX indicando el identificador de la referencia bibliográfica, por ejemplo: `\cite{aikg}`, que se se visualizaría como Dessi et al. [2020].

Cuando queramos citar más de un trabajo en un mismo punto del documento se deberán

añadir a la cita separados por comas, por ejemplo: `\cite{chen_2018, Qin_2020}`, que se visualizaría como Chen et al. [2018], Qin et al. [2020]. En estos casos es habitual añadir las citas en orden cronológico de más antigua a más reciente.

Tal y como está configurada esta plantilla, por defecto el comando `\cite` realizará una cita textual (`\citet`), es decir, la cita forma parte del texto. Este tipo de citas se suelen realizar en frases como:

“Algunos ejemplos de grafos de conocimiento se presentan en Ji et al. [2020]”

Cuando la cita no forma parte del texto si no que se utiliza para reforzar una afirmación realizada en una frase, se debe utilizar una cita entre paréntesis (o corchetes). Para ello se usa el comando `\citep`, por ejemplo:

“Existe una gran variedad de aplicaciones de los grafos de conocimiento [Ji et al., 2020].”

A.2. Figuras

Para añadir una figura al documento utilizaremos el entorno `figure`, indicando la posición que debe tener dicha figura en el documento (h: here, t: top, b: botom; p: page), se recomienda en lo posible evitar de “!” que ignora todos los ajustes de los parámetros. El orden en el que se indiquen cada una de las opciones se tendrá en cuenta para colocar la figura, es decir si se indicase el orden [htbp] la figura primero se intentará colocar en el lugar que ocupa en el documento, si no se puede se intentará colocar al inicio (top) de la página, en caso que tampoco sea posible se intentará colocar al final (bottom) de la página y, por último en caso que no sea posible ninguna de las anteriores se colocará al inicio de una página nueva.



Figura A.1: Esta figura tiene una descripción al pie muy larga, por lo que añadiremos un título breve utilizando para ello los corchetes tras el comando `\caption`. La etiqueta de la figura (label) se incluirá al inicio del flotante de la figura para que cualquier referencia cruzada (ref) a la misma lleve al inicio del flotante.

A.3. Tablas

Para añadir una tabla se utilizará el entorno `table`, indicando al inicio de la tabla el título de la misma utilizando el `caption`. Un ejemplo puede verse en la Tabla A.1.

Tabla A.1: Esta tabla presenta un ejemplo con tres columnas y formato formal.

Model	Accuracy	Precision	Recall	F1-score	AUC
Modelo 1	0,33	0,75	0,72	0,42	0,21
Modelo 2	0,01	0,63	0,60	0,50	0,10
Modelo 3	0,03	0,93	0,33	0,04	0,42

A.4. Código

Para añadir código en la memoria utilizaremos el paquete `listing`, que permite mostrar código formateado en diversos lenguajes (Java, Python, C, ...).

```

1 import numpy as np
2
3 def incmatrix(genl1, genl2):
4     m = len(genl1)
5     n = len(genl2)
6     M = None #to become the incidence matrix
7     VT = np.zeros((n*m,1), int) #dummy variable
8
9     #compute the bitwise xor matrix
10    M1 = bitxormatrix(genl1)
11    M2 = np.triu(bitxormatrix(genl2),1)
12
13    for i in range(m-1):
14        for j in range(i+1, m):
15            [r,c] = np.where(M2 == M1[i,j])
16            for k in range(len(r)):
17                VT[(i)*n + r[k]] = 1;
18                VT[(i)*n + c[k]] = 1;
19                VT[(j)*n + r[k]] = 1;
20                VT[(j)*n + c[k]] = 1;
21
22            if M is None:
23                M = np.copy(VT)
24            else:
25                M = np.concatenate((M, VT), 1)
26
27            VT = np.zeros((n*m,1), int)
28
29    return M

```

A.5. Listas

Como en cualquier procesador de textos debemos diferenciar dos tipos de listas. Las listas no numeradas (o de viñetas) se definirán mediante entornos `itemize`.

- Elemento no numerado
- Elemento no numerado
- Elemento no numerado

Las listas numeradas se definirán mediante entornos `enumerate`. En ambos casos, cada elemento de la lista se genera utilizando el comando `item`.

1. Elemento 1

Con el comando `\` podemos insertar saltos de línea sin cambiar de párrafo.

2. Elemento 2. Con el comando `\textbf` podemos **enfaticar con negrita** un texto y con el comando `\textit` podemos *enfaticar con itálica* un texto.

3. Elemento 3

A.6. Ecuaciones

Cuando una ecuación va a ser referenciada desde el texto, en principio más de una vez, será necesario nombrar o numerar dicha ecuación (ver Ecuación A.1). Para añadir una ecuación numerada utilizaremos el entorno `equation`.

$$\begin{pmatrix} w_1, & w_2, & \dots, & w_n \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} + b = 0 \rightarrow \mathbf{w}^t \mathbf{x} + b = 0 \quad (\text{A.1})$$

Cuando la ecuación no va a ser referenciada desde el texto, podemos utilizar la versión de entorno no numerada `equation*`

$$\frac{1}{2} \|\mathbf{w}\| = \frac{1}{2} \sqrt{\sum w_i^2}$$

Es también posible añadir ecuaciones en línea con el texto, para ello se debe incluir la expresión matemática en L^AT_EX encerrada entre símbolos de dólar. Por ejemplo, la ecuación anterior se podría representar también en línea: $\frac{1}{2} \|\mathbf{w}\| = \frac{1}{2} \sqrt{\sum w_i^2}$.